B.M.S. COLLEGE OF ENGINEERING BENGALURU

Autonomous Institute, Affiliated to VTU

OOMD Mini Project Report

Real-Time Bus Tracking System (RTBTS)

Submitted in partial fulfillment for the award of degree of

Bachelor of Engineering
in
Computer Science and Engineering

Submitted by:

Nithesh (1BM23CS218)
Pradeep G (1BM24CS414)
Prajwal K S (1BM24CS416)
S Mohammed Aiaz (1BM24CS418)

Department of Computer Science and Engineering
B.M.S. College of Engineering
Bull Temple Road, Basavanagudi, Bangalore 560 019

2025-26

B.M.S. COLLEGE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We, **Nithesh (1BM23CS218), Pradeep G (1BM24CS414), Prajwal K S (1BM24CS416), S Mohammed Aiaz (1BM24CS418)** students of 5th Semester, B.E, Department of Computer Science and Engineering, BMS College of Engineering, Bangalore, hereby declare that, this OOMD Mini Project entitled "**Real-Time Bus Tracking System (RTBTS)**" has been carried out in Department of CSE, B.M.S. College of Engineering, Bangalore during the academic semester August 2025-December 2025. I also declare that to the best of our knowledge and belief, the OOMD mini Project report is not from part of any other report by any other students.

Signature of the Candidate

Nithesh (1BM23CS218)
Pradeep G (1BM24CS414)
Prajwal K S (1BM24CS416)
S Mohammed Aiaz (1BM24CS418)

B.M.S. COLLEGE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the OOMD Mini Project titled "**Real-Time Bus Tracking System (RTBTS)"** has been carried out by **Nithesh (1BM23CS218), Pradeep G (1BM24CS414), Prajwal K S (1BM24CS416), S Mohammed Aiaz (1BM24CS418)** during the academic year 2025-2026.

Signature of the Faculty in Charge (Your Guide Name)

Table of Contents

Chapter 1: Problem Statement

Problem Description:

In small cities and tier-2 towns, public transport systems lack real-time tracking, causing inconvenience to commuters who face unpredictable bus schedules. This leads to overcrowding, wasted time, and reduced reliance on public transport. The problem is acute in cities with growing populations but limited digital infrastructure for transport management.

Impact / why this problem needs to be solved

Over 60% of commuters in small cities face delays due to lack of real-time information, reducing public transport usage and increasing private vehicle dependency, which worsens traffic and pollution. A solution would enhance commuter experience and promote sustainable transport.

Expected outcomes

• A mobile app or web platform integrating gps-based real-time tracking of buses.
• Display estimated arrival times and route information.
• Optimized for low-bandwidth environments to ensure accessibility in smaller towns.

Relevant stakeholders / beneficiaries

• Commuters
• Local Transport Authorities
• Municipal Corporations

Supporting data

• Urban mobility india report 2024, emphasizing transport inefficiencies in tier-2 cities.

Chapter 2: Software Requirement Specification

Real-Time Bus Tracking System (RTBTS):

1. Introduction

1.1 Purpose of this Document

The purpose of this document is to outline the requirements and specifications for the development of a Real-Time Bus Tracking System. It will provide a clear understanding of the project objectives, scope, and deliverables for developers, stakeholders, and transport authorities.

1.2 Scope of this Document

This document defines the overall working and main objectives of the Real-Time Bus Tracking System. It includes a description of the project's purpose, estimated development cost, and time required for implementation.

1.3 Overview

The Real-Time Bus Tracking System is a software solution designed to improve public transport efficiency by integrating **GPS-based bus tracking**. The system will display **real-time bus locations**, **estimated arrival times (ETA)**, and **route information** through a **mobile app and web portal**. It is particularly targeted for small and tier-2 cities, where lack of reliable transport information causes delays and inconvenience to commuters.

2. General Description

The Real-Time Bus Tracking System will serve commuters, transport authorities, and municipal corporations. It will allow users to check bus locations, track estimated arrival times, and receive alerts for route updates. Transport authorities will be able to manage fleets, monitor punctuality, and optimize routes. The system will be designed for low-bandwidth environments to ensure accessibility in smaller towns.

3. Functional Requirements

3.1 Bus Tracking:

- The system **shall** track buses in real-time using GPS devices.
- The system **shall** display the current location of buses on a map.

3.2 ETA Calculation:

- The system **shall** calculate and display estimated arrival times for each stop.
- The system **should** adjust ETAs dynamically based on traffic and delays.

3.3 Route Management:

- The system **shall** provide detailed route information, including stops and timings.
- The system **should** allow authorities to update routes in case of diversions.

3.4 Commuter Services:

- The system **shall** allow commuters to search for buses by route or stop.
- The system **shall** provide notifications for bus arrivals, delays, or cancellations.

3.5 Reporting:

- The system **shall** generate daily/weekly reports for authorities (fleet performance, punctuality, passenger usage).

4. Interface Requirements

4.1 User Interface:

- Mobile app for commuters with **GPS-based map view.**

- Web portal/dashboard for transport authorities.
- User-friendly design optimized for low-bandwidth networks.

4.2 Integration Interfaces:

- Integration with **GPS devices** installed in buses.
- Integration with **SMS/notification APIs** for commuter alerts.

5. Performance Requirements

5.1 Response Time:

- The system **shall** update bus location data every 10–15 seconds.

5.2 Scalability:

- The system **shall** handle at least 10,000 concurrent users in tier-2 cities.

5.3 Data Integrity:

- The system **shall** ensure accurate and reliable GPS data synchronization.

6. Design Constraints

6.1 Hardware Limitations:

- The system **shall** be compatible with standard GPS devices and smartphones.

6.2 Software Dependencies:

- Use a relational database (MySQL/PostgreSQL).
- Backend: Node.js/Java/Python; Frontend: React/Flutter for mobile.

7. Non-Functional Attributes

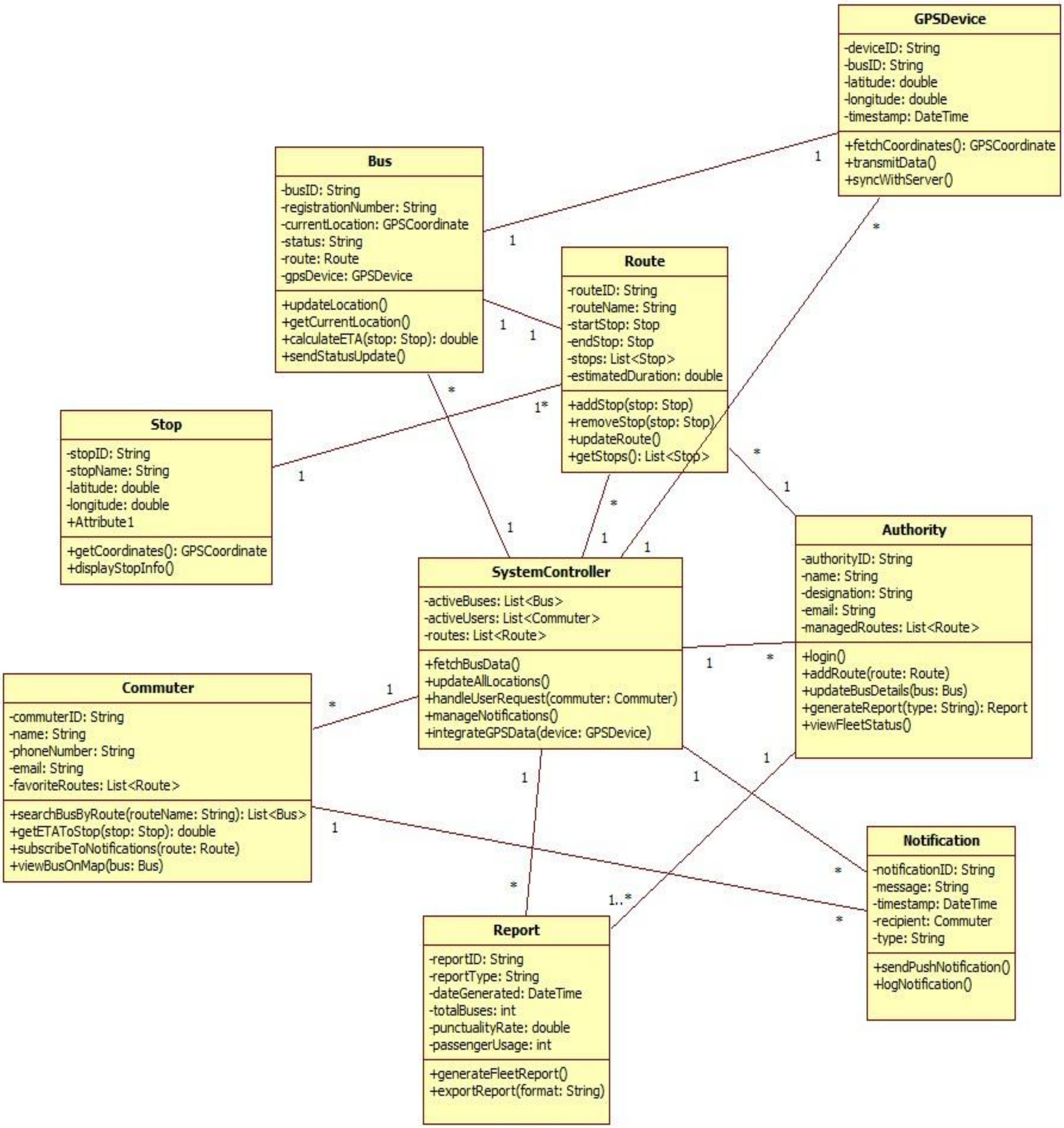
- **Security:** Data encryption for commuter information and secure APIs.
- **Reliability:** High uptime (99.9%) to ensure commuter trust.
- **Scalability:** Support for more buses and cities in the future.
- **Portability:** Accessible across Android, iOS, and web browsers.

- **Usability:** Simple interface for commuters with minimal data usage.
- **Compatibility:** Support popular browsers and devices.

8. Preliminary Schedule and Budget

- **Timeline:** Estimated **6 months** (Requirement analysis – 3 weeks, Design – 4 weeks, Development – 12 weeks, Testing – 4 weeks, Deployment – 3 weeks).
- **Budget:** Approximately **$60,000 – $100,000**, depending on GPS integration and scale of deployment.

Chapter 3: Class Modeling



1. RTBTS (Central System Controller)

The RTBTS acts as the core orchestrator of the entire real-time bus tracking environment.
It manages system initialization, coordinates interactions between buses, GPS devices, commuters, and authority controllers, and ensures seamless data flow throughout the ecosystem. By centralizing operations—such as location updates, ETA calculations, route management, and notification distribution—it guarantees synchronized and highly efficient system performance. As the "brain" of the system, it ensures that real-time information is processed, validated, and delivered accurately to users, forming the foundation upon which the entire application operates.

2. Bus

The **Bus** class represents each individual vehicle tracked by the system.
It maintains critical real-time attributes including the current location, assigned route, GPS device linkage, movement status (on-time, delayed, halted), and position history. The class continuously receives GPS data and communicates with RTBTS to update its status, enabling ETA calculations and route monitoring. By modeling each bus as an intelligent digital entity, the system can perform fine-grained tracking, detect deviations, and deliver reliable location information to commuters and authorities.

3. GPSDevice

GPSDevice class serves as the technological backbone that captures live bus coordinates.
It continuously gathers latitude, longitude, and timestamp data and transmits them to the RTBTS. This class ensures high accuracy under varying conditions—movement, network fluctuations, and signal loss. As the core sensor component, it is crucial for real-time tracking, ETA accuracy, route adherence verification, and notification triggers. Without this class, the system would be unable to function as a real-time monitoring platform.

4. Commuter

The **Commuter** class represents every user interacting with the system.
It handles search requests, bus tracking, route browsing, subscription to alerts, and viewing bus details on the map. This class defines how the passenger receives

meaningful information: nearby stops, ETA, live tracking, and notifications when the bus approaches. By storing preferences like favorite routes and recently tracked buses, it enhances usability and personalization, ensuring a seamless passenger experience.

5. Route

The **Route** class models complete bus routes including start point, end point, intermediary stops, and estimated durations.
It organizes the structure of how buses move across the city and enables the system to compute ETA, detect delays, suggest alternative routes, and manage schedule deviation. Route objects form the blueprint used by both commuters and controllers, ensuring consistency and precision across all operations.

6. Stop

The **Stop** class defines each bus stop with geographic coordinates, stop names, and linking routes.
It plays a critical role in calculating ETA, determining the bus-near-stop event for notifications, mapping commuter-entered queries to actual locations, and generating route-based visualizations. Stops also assist in error detection—such as skipped stops or unplanned halts—allowing the system to identify operational anomalies.

7. NotificationService

The NotificationService is responsible for generating and delivering timely alerts to commuters.
It monitors bus positions, checks subscription data, compares ETA values, and triggers notifications if a bus is near a stop or delayed. This class ensures passengers receive actionable, timely updates, improving reliability and commuter satisfaction. Through scheduled checks and event-driven triggers, it ensures optimal communication between system intelligence and end-users.

8. ReportEngine

The ReportEngine compiles operational information to produce meaningful insights for authorities.
It analyzes logs, bus punctuality, route performance, system errors, and commuter usage patterns to generate daily, weekly, or monthly reports. Authorities rely on this class for planning improvements, identifying inefficiencies, optimizing routes, and

enhancing overall public transportation strategy. It plays a major role in long-term optimization and accountability.

9. Authority

The **Authority** class represents system administrators or transportation officials. It grants them the ability to manage routes, monitor fleet performance, generate system reports, and modify operational rules. Authorities can track delays, update stops, address commuter complaints, and oversee system health. This class ensures that the transport department maintains full control over the system, guaranteeing reliability and transparency.

10. Notification (Message Object)

This class represents a single notification event generated for a commuter. It stores the message content, timestamp, recipient, and type of alert (bus near stop, delay, ETA update). These objects are logged for auditing, used by the ReportEngine for analytics, and help ensure accountability by maintaining a track record of all communication events in the system.

11. Log / ErrorLog (if present in your class diagram)

This class ensures all system activities—GPS failures, delayed notifications, route mismatches—are recorded systematically.
It serves as an audit trail, provides debugging support, and helps the Authority and ReportEngine identify recurring issues. Its presence is crucial for maintaining reliability, transparency, and long-term system health.

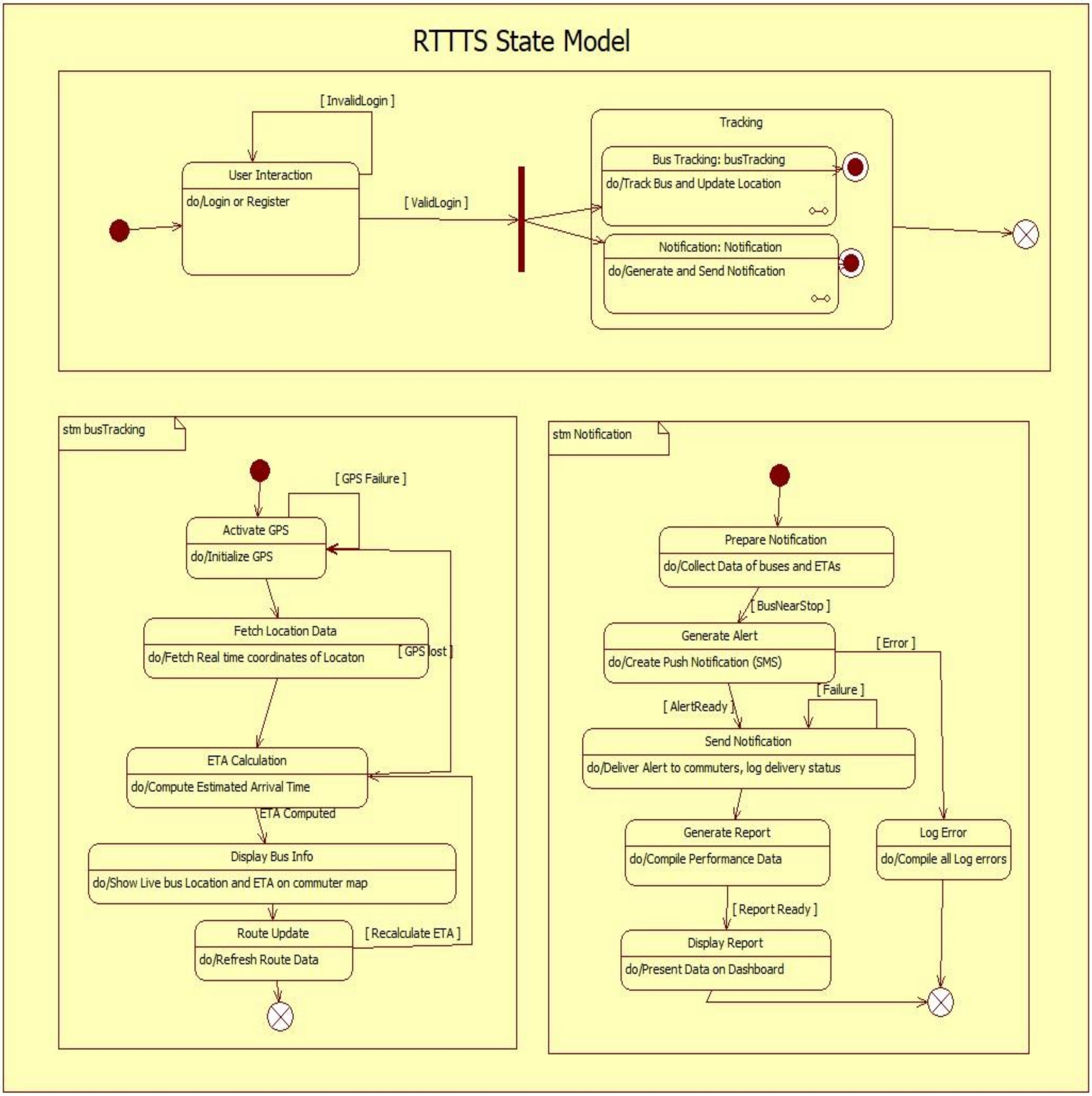
This system brings together **real-time sensors, digital route modeling, user-centric tracking, automated notifications, and administrative oversight** to create a reliable, scalable, and commuter-friendly transportation monitoring platform.

It ensures:

- ✔ **Real-Time Accuracy**
- GPS + Bus + RTBTS
- ✔ **Commuter Convenience**

- Commuter + NotificationService
- ✔ **Administrative Control**
- Authority + ReportEngine
- ✔ **System Health & Transparency**
- Logs + Reports + Route Management
- ✔ **Scalability**
- Modular class architecture allows easy expansion.

Chapter 4: State Modeling



The **Real-Time Bus Tracking System (RTBTS)** enables continuous monitoring of bus locations, predicting arrival times, notifying passengers, and keeping system data updated. The state diagram captures the complete lifecycle of RTBTS—from startup to GPS tracking, route processing, alert handling, and maintenance.
Each state and event ensures the system operates seamlessly for passengers, bus operators, and administrators.

1. System Initialization

Load System Modules

- Entry state where all required components (GPS service, map routing engine, notification module, database connections) are loaded.
- Ensures the foundational environment is set before real-time tracking begins.

Validate System

- Performs security and integrity checks on GPS connections, backend servers, and route database.
- Ensures safe, stable operations before moving into live mode.

2. Real-Time Tracking

GPS Tracking Active

- Activates GPS modules installed in the bus.
- Continuously collects location coordinates at regular intervals.

Receive GPS Data

- GPS packets containing latitude, longitude, timestamp, and bus ID are received.

- This data becomes the basis for route comparison and ETA calculations.

3. Route Processing

Process Bus Location

- Validates the GPS data, checks accuracy, and filters noise.
- Matches the location to its nearest road segment or route point.

Update Live Route Status

- Updates the real-time status of each bus:
  - Current location
  - Current stop
  - Direction
  - Delay or ahead-of-time indicator
- Ensures passengers always see accurate movement data.

4. ETA Prediction & User Interaction

Compute ETA

- Calculates Estimated Time of Arrival using:
  - Current speed
  - Traffic conditions
  - Distance to stop

    - Historical travel patterns
- Ensures users get realistic arrival predictions.

Send Notifications

- Sends alerts to passengers about:
    - Bus approaching
    - Delay alerts
    - Route deviation
- Ensures timely and relevant updates for passengers.

5. Alert State

Route Deviation Alert

- Triggered when the bus deviates significantly from the assigned route.
- Helps operators take corrective action quickly.

Breakdown / Emergency

- Activated if the system detects:
    - Bus breakdown
    - Driver emergency alert
    - GPS device failure
- Sends alerts to admins/operators for intervention.

6. Admin Interaction

Acknowledge Alert

- Admin or operator confirms receiving deviation or breakdown notifications.
- Begins corrective actions like rerouting or dispatching another bus.

Update Bus Information

- Admin can update:
  - Bus routes
  - Driver details
  - Assigned schedules
- Ensures the system reflects correct operational data.

7. System Maintenance

Archive Old Data

- Periodically moves old tracking and route data into archives to maintain performance.

Backup & Restore

- Creates system backups and restores services if data errors or failures occur.

Update Map & Routes

- Updates map data, GPS firmware, and routing algorithms.
- Ensures system accuracy with evolving routes and traffic conditions.

Initialize System

- Triggers module loading and system validation for a smooth startup.

GPS Signal Received

- Moves the system from idle to processing live tracking data.

Valid Location Update

- Indicates new GPS coordinates are ready for route processing and ETA calculations.

Route Deviation Detected

- Event raised when the bus moves off its permitted path by predefined margins.

Emergency Signal Triggered

- Generated if the driver presses an emergency button or GPS device fails.

ETA Required

- Triggered when a passenger opens the app or requests arrival time.

Notification Trigger

- Sends push notifications for approaching buses, delays, or alerts.

Admin Update Request

- When an admin updates route information or bus schedules.

Maintenance Trigger (Scheduled/Manual)Starts actions like data cleanup, backups, or firmware updates.

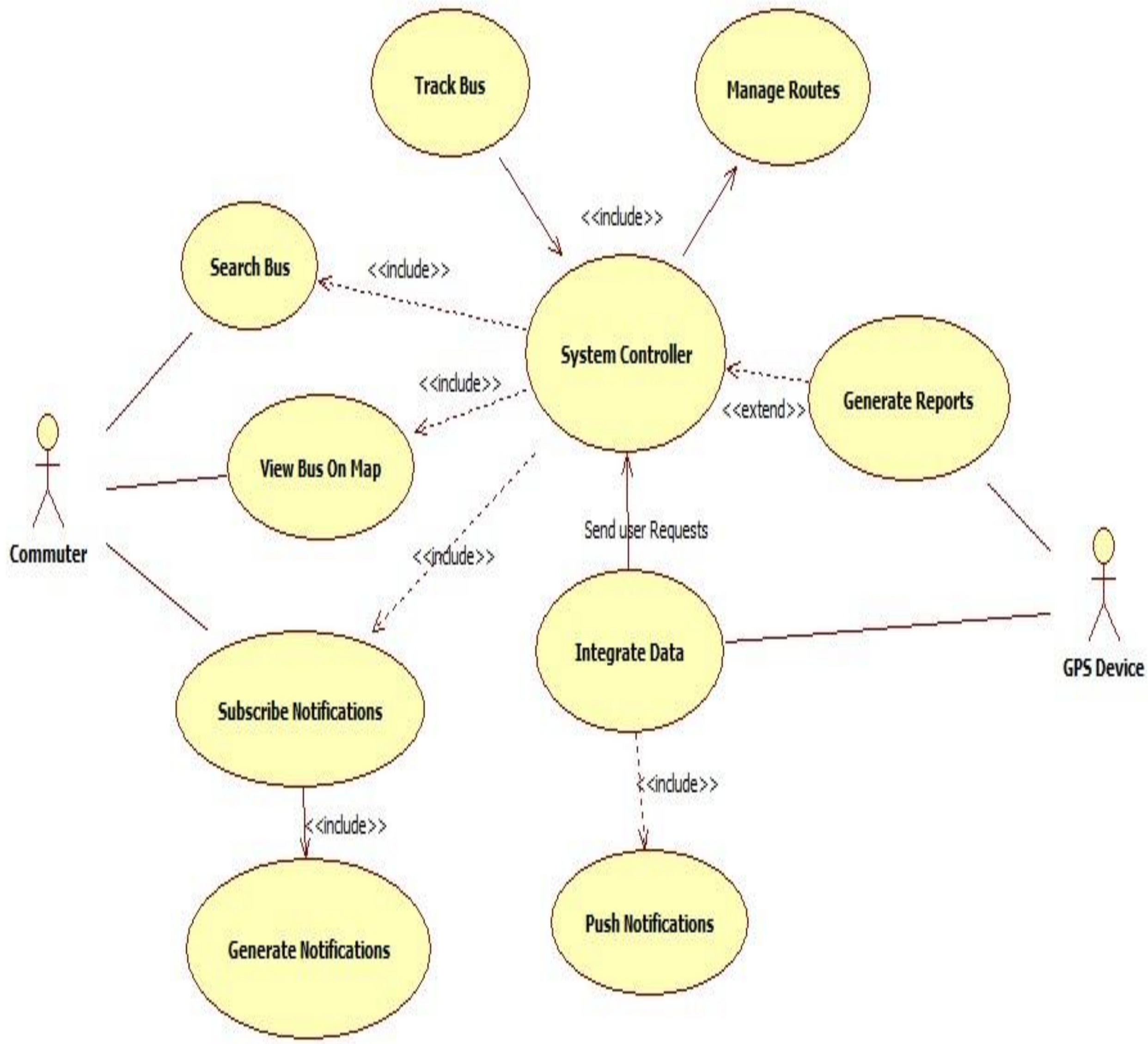
The **RTBTS state diagram** outlines a complete functional cycle:

1. **Initialization** loads system modules.
2. **Tracking mode** captures real-time GPS data.
3. **Route pro**
4. **cessing** maps bus locations to correct routes.
5. **ETA prediction** informs passengers of arrival times.
6. **Alerts** respond to emergencies or route deviations.
7. **Admin interaction** ensures operational control.
8. **Maintenance** keeps the system rliable and updated.

Chapter 5: Interaction Modeling

USE CASE:

.



RTBTS — Relevance of Each Use Case

System Initialization

Marks the beginning of system operations where the administrator starts the RTBTS platform.
It ensures all tracking modules, GPS connections, route databases, and notification services are fully operational before real-time monitoring begins.

System Shutdown

Allows administrators to safely terminate the system.
Shuts down ongoing GPS streams, closes database connections, and ensures that no tracking or notification tasks are abruptly interrupted.

Real-Time Bus Tracking

Continuously monitors bus locations through GPS inputs.
This use case forms the core of RTBTS, updating live bus coordinates, detecting movement patterns, and enabling accurate ETA calculations.

Route Monitoring

Checks whether buses are following their assigned routes.
Identifies route deviations early and ensures operators can intervene before service disruptions affect passengers.

ETA Prediction

Computes the Estimated Time of Arrival for each bus at every stop.
Enhances user experience by giving accurate, real-time predictions based on speed, distance, traffic, and historical patterns.

Passenger Notification

Sends real-time alerts to passengers waiting at stops.
Notifies them about bus arrival, delays, route changes, or cancellations, ensuring timely communication and better convenience.

Bus Operator Interaction

Operators monitor bus behavior, acknowledge alerts, and manage route changes.
This use case ensures operational control and quick responses to deviations, emergencies, or delays.

Emergency Alert Handling

Triggered when the system detects emergencies like breakdowns, accidents, or GPS failure.
The system alerts administrators and operators, helping them take corrective action to maintain safety and service continuity.

Update Bus / Route Data

Allows administrators or operators to update:

- Bus schedules
- Assigned drivers
- Routes
- Stops
- Timings
  This ensures that RTBTS always reflects the latest operational data.

Data Archiving

Moves old GPS logs, route history, and bus performance data to an archive.
Improves system efficiency by clearing outdated records without losing historical insights.

Backup and Recovery

Creates system backups and enables rapid recovery during failures.
Ensures uninterrupted service and prevents loss of operational or historical data.

Feedback Submission

Passengers and operators submit feedback on service quality, delays, or technical issues.
This promotes continuous improvement and enhanced public-service accountability.

Map & Model Update

Periodically updates:

- Map data
- Traffic models
- Routing algorithms
  Ensures the system adapts to real-world changes such as new roads, diversions, or traffic behavior patterns.

Interaction of Actors

Administrator

- Starts and shuts down the system
- Updates bus data, schedules, and routes
- Oversees emergency situations
- Performs maintenance, backup, and recovery
  Ensures system stability and accuracy.

Bus Operator

- Monitors real-time bus positions
- Confirms route deviations or emergencies
- Updates bus availability or driver information

Ensures smooth transit operations.

Passenger

- Receives notifications
- Tracks buses in real time
- Checks ETA predictions
- Submits service feedback
 Improves daily commuting with accurate and timely updates.

GPS Device

- Continuously sends bus coordinates
- Serves as the backbone of tracking operations
 Without accurate GPS data, RTBTS cannot function.

Notification System

- Pushes alerts to passengers and operators
- Notifies delays, arrivals, emergencies, and route changes
 Ensures timely and reliable communication.

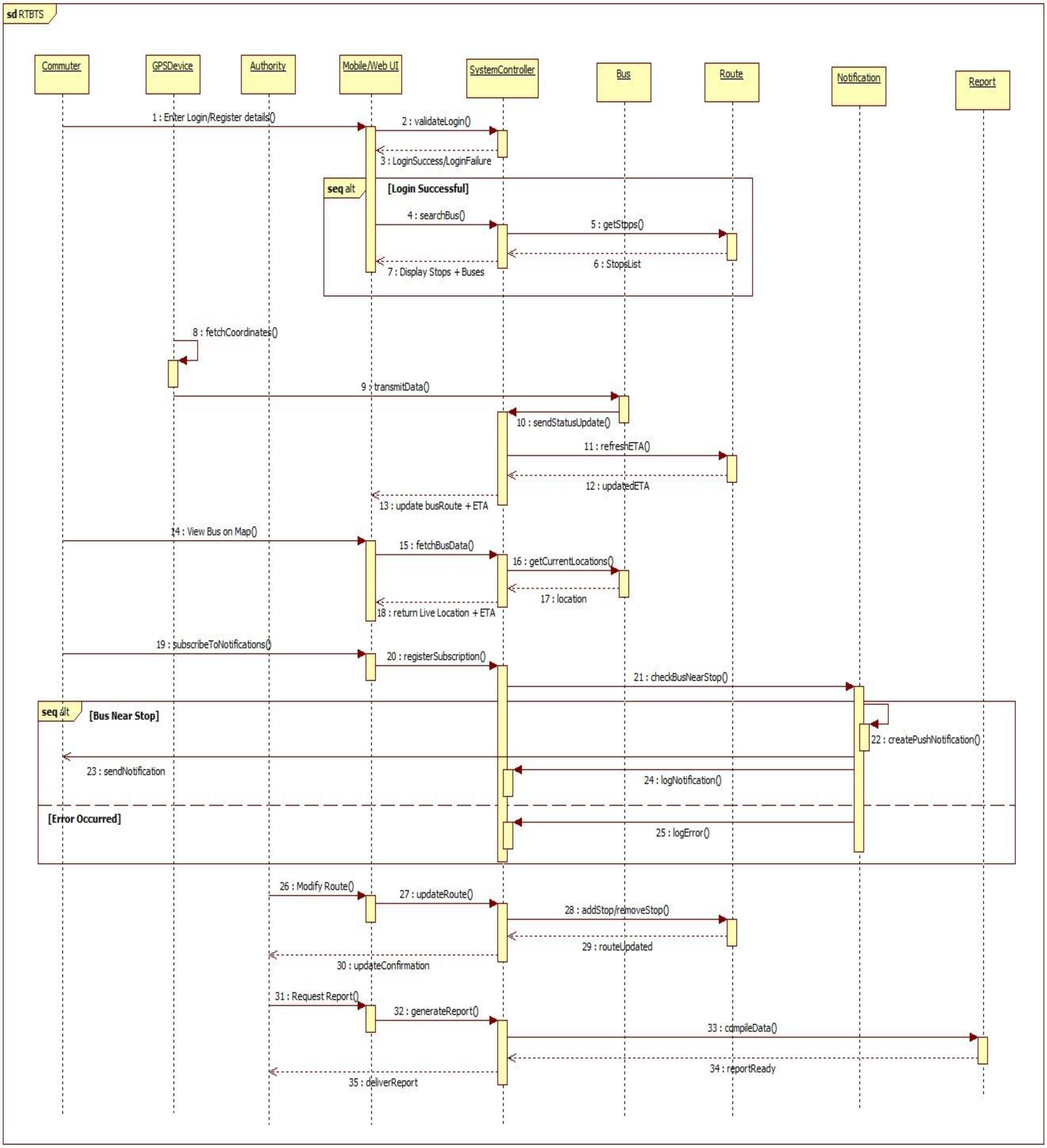
Route / Map Database

- Stores route paths, stop locations, and times
- Used for ETA prediction, deviation detection, and navigation
 Provides the intelligence required for reliable tracking.

Central Server

- Processes GPS data
- Computes ETAs
- Handles archiving, backup, and recovery
 Acts as the computational core of the RTBTS.

Sequence Diagram:



RTBTS – Sequence Diagram Explanation

The sequence diagram for the **Real-Time Bus Tracking System (RTBTS)** illustrates how administrators, bus operators, passengers, GPS devices, and system modules interact across different processes such as system startup, real-time tracking, ETA computation, notifications, route monitoring, emergency handling, and feedback submission.
It highlights how each actor communicates with system components to ensure smooth, accurate, and reliable public transportation management.

🧑‍💼 Administrator

1. Initialize System

The administrator initiates the RTBTS platform by activating modules such as:

- GPS data receiver
- Route database
- Notification engine
- Tracking and ETA computation services

This allows the system to begin processing live bus locations.

2. Monitor System Status

The administrator checks:

- Live GPS data flow
- Tracking accuracy
- Server load
- Route monitoring alerts

Ensures all components operate correctly and prepares the platform for real-time passenger updates.

3. Update Bus/Route Data

When buses or schedules change, the administrator updates:

- Bus details
- Assigned routes
- New stops or revised timings

These updates reflect immediately in tracking and ETA calculations.

4. Oversee Emergency Logs

When emergency alerts are triggered (breakdowns, accidents), the administrator reviews them and coordinates with bus operators.

🚌 Bus Operator

1. Track Bus & View Alerts

Operators view the real-time status of their buses:

- Current location
- Speed
- ETA
- Route deviation alerts

This helps them maintain schedule accuracy.

2. Acknowledge Route Deviations

If a bus deviates from its planned route:

- The system sends a deviation alert

- Operator acknowledges it
- Takes corrective action or updates the route if needed

3. Respond to Emergency Notifications

When the system detects issues such as breakdowns or abnormal stops:

- Operator receives an emergency alert
- Identifies the cause
- Communicates with the admin or driver to take action

4. Update Operational Information

Operators may update bus status (in service, out of service), ensuring accuracy in tracking.

👥 Passenger

1. Request Bus Location / ETA

Passengers interact with the system (via app or display boards) to:

- Check live bus position
- View ETA at their stop
- Track the bus approaching in real time

The system responds using the latest GPS data.

2. Receive Notifications

Passengers receive timely updates such as:

- “Bus arriving soon” alerts
- Delay notifications

- Route change or cancellation alerts

Improves commuter convenience and planning.

3. Submit Feedback

Passengers can submit feedback regarding:

- Delays
- Route issues
- Bus condition
- Tracking accuracy

The system logs the feedback to improve service quality.

🛰️ Key System Processes

1. System Initialization

- GPS receivers activate
- Tracking algorithms load
- Map and route databases are read
- Notification service starts

Prepares RTBTS for real-time operations.

2. Real-Time Bus Tracking

GPS devices installed on buses send live coordinates to the system.
The tracking module processes:

- Location updates

- Speed
- Direction
- Stop detection

Updates appear for operators and passengers instantly.

3. Route Monitoring

The system analyzes GPS coordinates to check if buses follow assigned routes.
If deviations occur:

- Alerts are triggered
- Operators receive notifications
- Admin may intervene

Ensures service reliability.

4. ETA Prediction

Using real-time GPS data, traffic patterns, and route distance, the system:

- Calculates Estimated Time of Arrival
- Updates ETAs dynamically
- Sends precise predictions to passengers

Enhances planning and reduces waiting frustration.

5. Notification Handling

Notifications are sent to:

- Passengers (arrival, delay, cancellation)
- Operators (route deviation, emergency)

- Administrators (critical system events)

Keeps all stakeholders updated.

6. Emergency Alert Handling

Triggered when the bus sends signals such as:

- Unusual stop duration
- Breakdown
- Accident
- GPS failure

System notifies operator and admin, enabling rapid intervention.

7. Feedback Logging

Both passengers and operators can submit feedback, which the system:

- Validates
- Logs
- Forwards to administrators

Promotes continuous system improvement.

🔔 Notification and Updates

Notifications for Bus Movements

The system continually alerts passengers and operators about:

- Bus arrival
- Route deviations

- Speed abnormalities
- Unexpected stoppages

ETA and Delay Updates

Passengers constantly receive updated ETAs based on real-time changes.

Emergency Alerts

Operators and administrators are immediately notified of:

- Breakdowns
- Accidents
- GPS malfunctions

Feedback & Report Status Updates

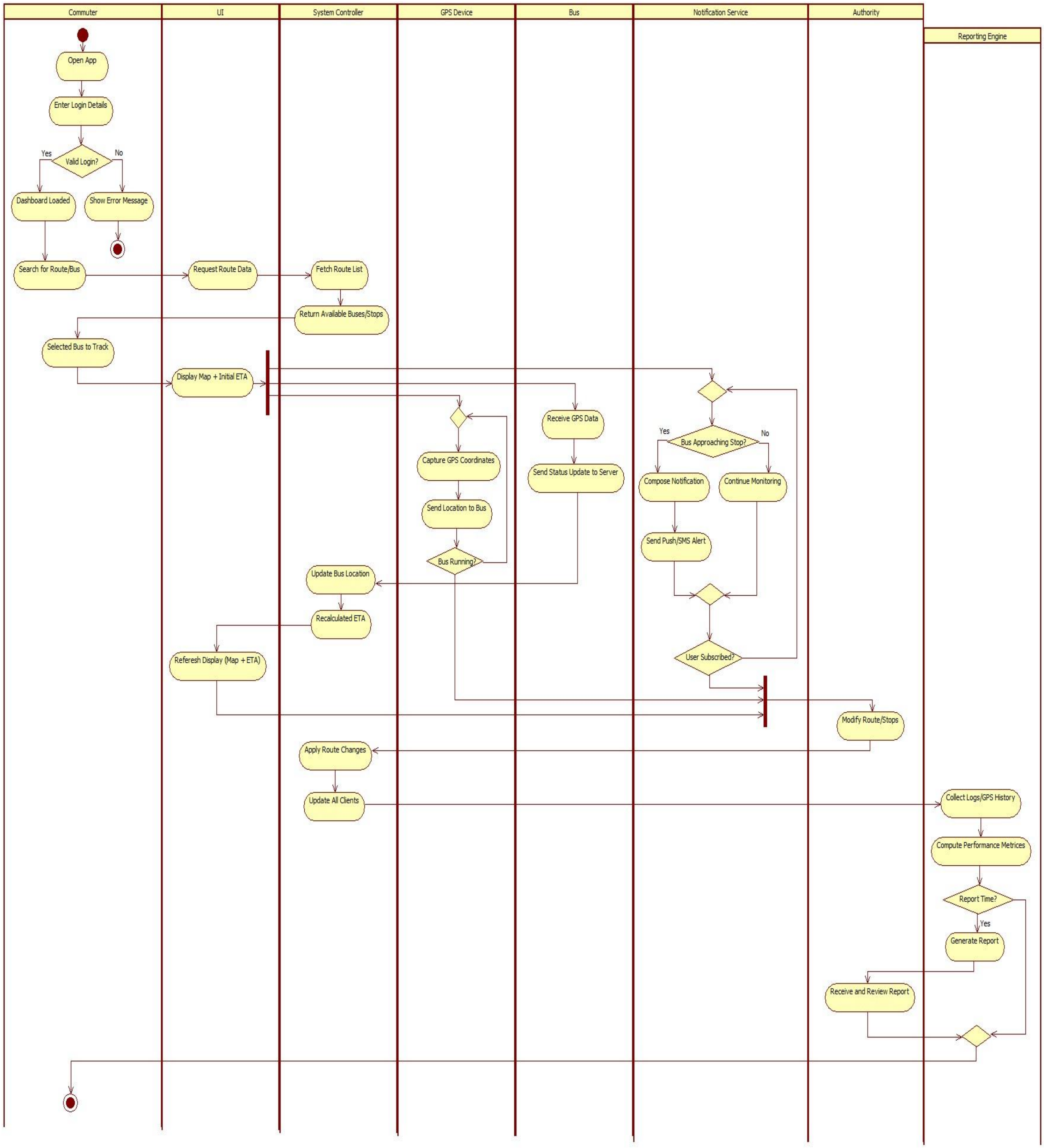
Passengers receive acknowledgment for:

- Feedback submissions
- Issue reports

Operators get status updates for:

- Emergency resolutions
- Route correction confirmations

Activity Diagram:



Activity Diagram Explanation for RTBTS

The activity diagram represents the functional flow of the **Real-Time Bus Tracking System (RTBTS)** across its major components: GPS Device, Tracking Engine, ETA Calculation Module, Route Monitoring Module, Notification System, Bus Operator Interaction, and Passenger Interface.

It provides a clear view of how the system starts, how activities flow through different modules, how the process branches into parallel actions, and how all paths finally merge to deliver real-time tracking and passenger updates.

Swimlanes

1. GPS Device Swimlane

Responsible for collecting real-time bus location data and transmitting it to the system. This lane is focused on:

- Device initialization
- Capturing GPS coordinates
- Sending continuous location updates
- Logging data transmission events

It forms the foundation for all tracking operations.

2. Tracking Engine Swimlane

Handles the core location processing tasks:

- Receiving GPS data
- Updating bus location on the system
- Validating GPS consistency
- Detecting stops or movement patterns

This lane ensures the system always has the current, verified position of each bus.

3. ETA Calculation Module Swimlane

Processes live location and route data to compute:

- Estimated Time of Arrival (ETA)

- Delay predictions
- Real-time travel speed analysis

This lane ensures passengers receive accurate and dynamically updated arrival times.

4. Route Monitoring Swimlane

Focuses on ensuring buses follow designated routes:

- Compare live GPS path with official route
- Detect and log route deviations
- Trigger route alerts for bus operators and administrators

This lane plays a key role in maintaining operational discipline and system reliability.

5. Notification System Swimlane

Responsible for communication and system alerts:

- Generating notifications for passengers
- Sending alerts to bus operators about deviations or delays
- Escalating emergencies to administrators

This lane ensures all stakeholders are promptly informed of changes, issues, or updates.

6. Bus Operator Interaction Swimlane

Shows the operator-side actions of RTBTS:

- Receiving route deviation alerts
- Checking bus status
- Acknowledging notifications
- Managing or updating emergency conditions

This lane handles operational decision-making and timely corrective actions.

7. Passenger Interface Swimlane

Covers user-facing actions:

- Requesting live bus location
- Viewing ETA
- Receiving arrival or delay notifications
- Submitting feedback

This lane ensures passengers receive accurate information and can interact easily with the system.

8. Splitting and Merging of Control

Splitting

After the GPS device sends a location update, the flow **splits into three parallel activities**:

1. **Tracking Engine**
   Processes and updates the bus's real-time location.
2. **ETA Calculation Module**
   Computes dynamic ETAs for upcoming bus stops.
3. **Route Monitoring Module**
   Compares the bus's path with the scheduled route and checks for deviations.

All three operate simultaneously to ensure the system remains real-time and accurate.

Merging

Once location is processed, ETA is calculated, and route status is evaluated, the control **merges** in the:

Notification System

Here the system:

- Generates passenger alerts
- Notifies operators/admins of deviations
- Sends "bus arriving soon" messages
- Issues emergency alerts if applicable

Finally, the flow moves into the Passenger and Operator swimlanes where the notifications are consumed.

9. Explanation of the Diagram

Below is the detailed activity description for each swimlane:

Actions of the GPS Device

Initialize GPS Unit

Activates tracking hardware and begins communication with the system.

Capture Coordinates

Samples latitude, longitude, speed, and direction.

Transmit Location

Continuously sends real-time location data to the Tracking Engine.

Log GPS Activity

Stores GPS signal quality, timestamp, and transmission logs.

Actions of the Tracking Engine

Receive GPS Data

Collects raw coordinates from the GPS device.

Update Bus Location

Places the bus on the map and updates its real-time movement.

Validate Signal

Checks for GPS anomalies such as:

- Sudden jumps
- Missing data
- Out-of-range coordinates

Log Tracking Activity

Maintains history of movement patterns for analysis.

Actions of ETA Calculation Module

Fetch Route and Traffic Data

Collects route distance and traffic conditions.

Compute ETA

Calculates arrival times based on:

- Current speed
- Distance to next stop
- Delay history

Update ETA Display

Sends updated ETAs to passenger apps and display boards.

Log ETA Activity

Records ETA changes and delay predictions.

Actions of Route Monitoring Module

Compare Path with Route

Checks if the bus remains on its planned course.

Detect Route Deviations

Triggers alerts when:

- Bus takes an alternate path
- Bus skips stops
- Unauthorized detours occur

Flag Irregularities

Logs deviation events for administrators.

Send Route Status

Forwards results to the Notification System.

Actions of Notification System

Generate Notifications

Produces alerts based on:

- ETA changes
- Route deviations
- Emergency conditions

Notify Passengers

Sends smartphone or display-board alerts such as:

- “Bus arriving in 2 minutes”
- “Bus delayed by 10 minutes”

Notify Operators

Provides alerts about operational issues.

Escalate Incidents

Critical alerts are forwarded to administrators.

Actions of Bus Operator Interaction

Receive Deviation Alerts

Operator gets notified of the bus going off-route.

Check Bus Status

Reviews real-time location, ETA, and delay information.

Acknowledge Notifications

Confirms receipt of system alerts.

Handle Emergencies

If a breakdown or accident occurs, operator updates the system.

Actions of Passenger Interface

Request Bus Location

Passenger searches for bus and route details.

View ETA

Displays updated arrival times and delays.

Receive Alerts

Passenger gets:

- Arrival alerts
- Delay notifications
- Route change updates

Submit Feedback

Provides feedback about service quality or issues.

The RTBTS activity diagram illustrates:

- How GPS data triggers multi-step processing
- How parallel operations (tracking, monitoring, ETA) ensure accuracy
- How flows merge into notification and user interaction
- How each module contributes to providing reliable real-time bus tracking

The structured swimlanes and control flows clearly depict how the system supports operators, administrators, and passengers with continuous, accurate, and timely information.

Chapter 6: UI Design with Screenshots

Figure 1: Home Page

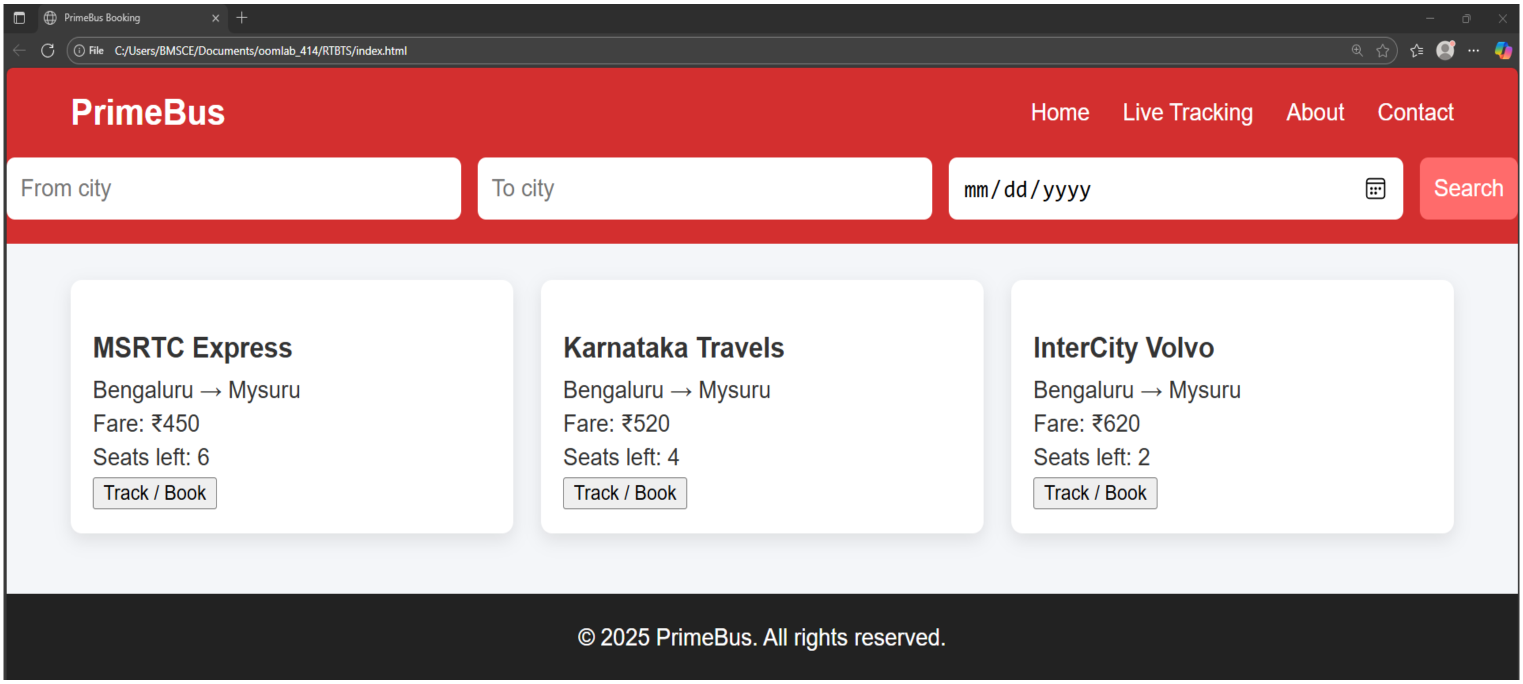


Figure 2 : Live Bus Tracking Page

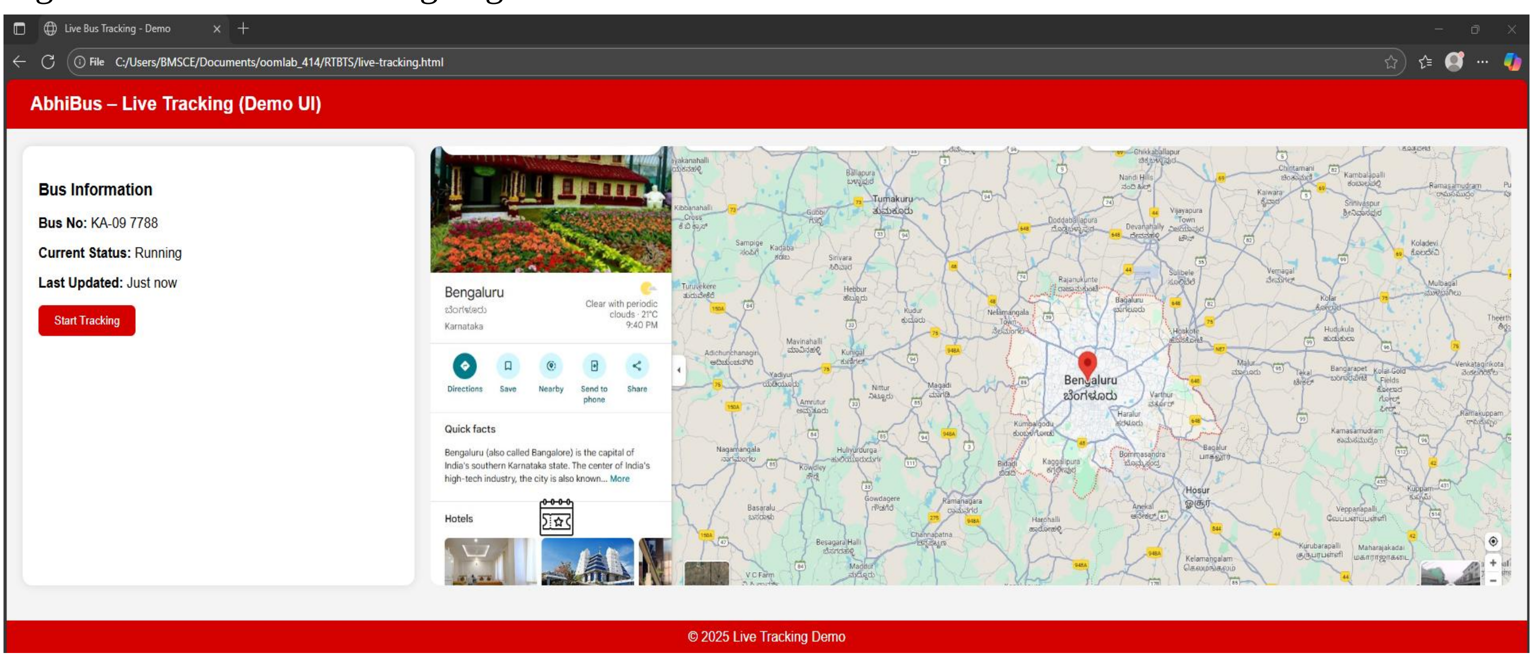


Figure 3: About Page

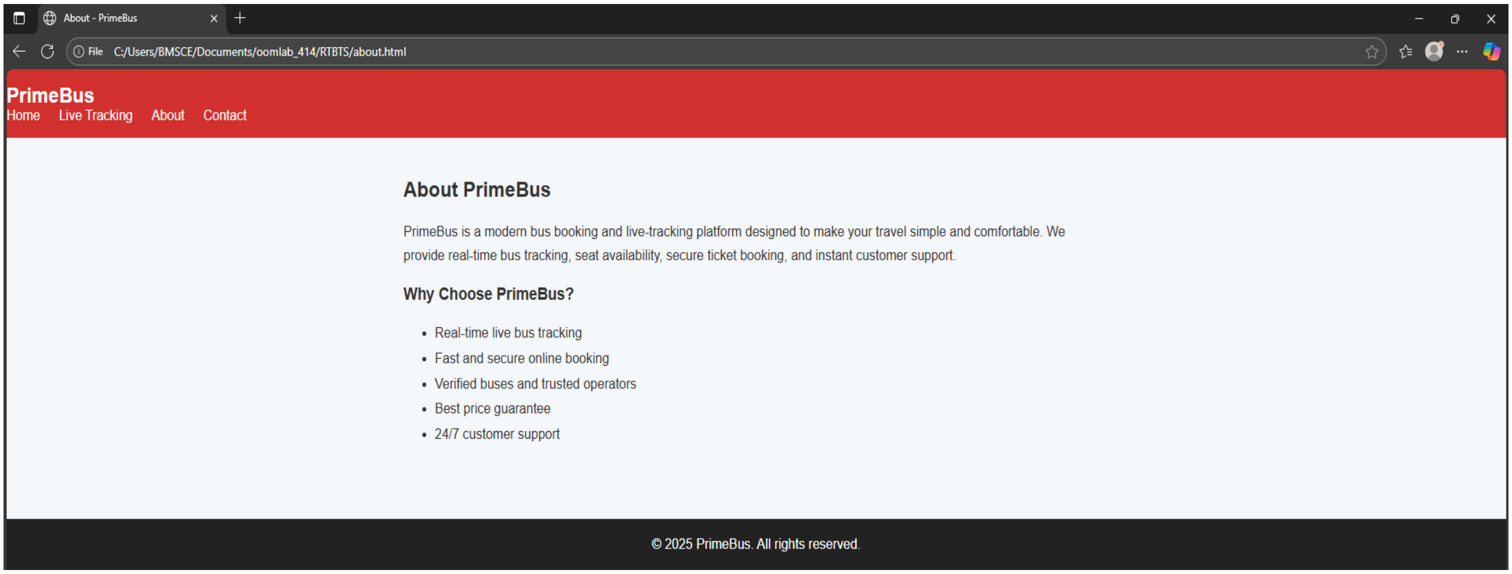
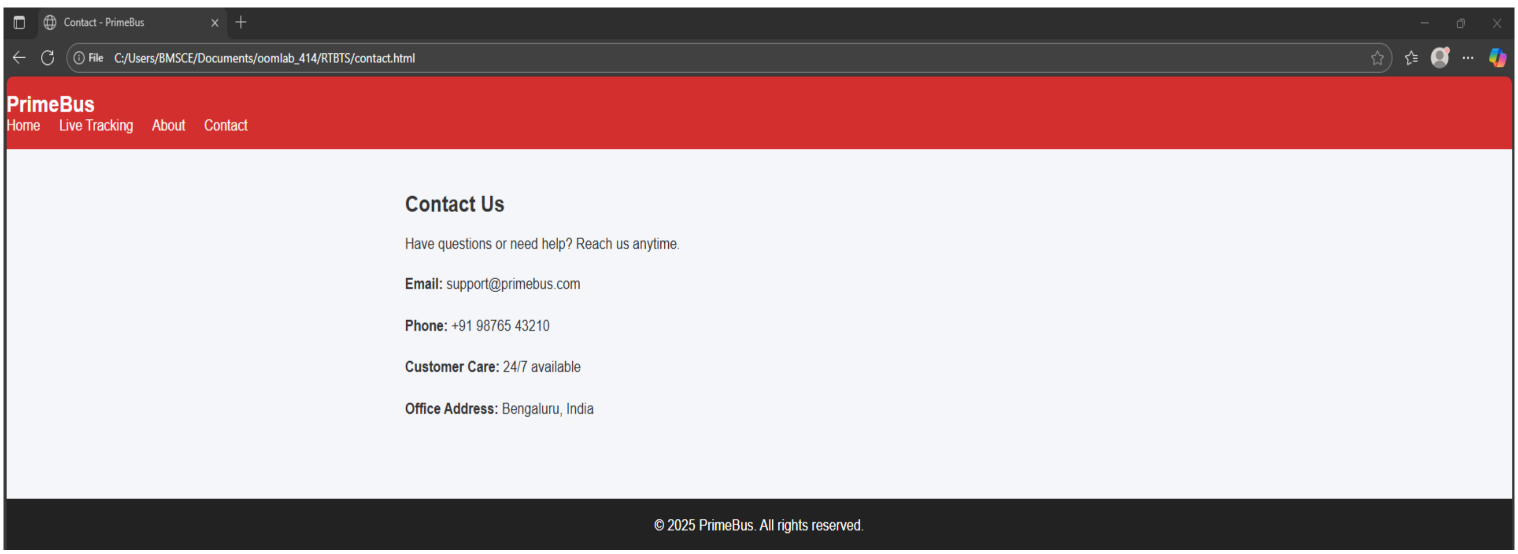


Figure 4: Contact Page



Conclusion

The Real-Time Bus Tracking System (RTBTS) addresses major challenges in modern public transportation by enabling continuous bus monitoring, accurate ETA prediction, route deviation detection, and seamless communication between passengers, operators, and administrators. The system is designed to integrate effectively with GPS hardware, route databases, traffic data sources, and notification services, ensuring that all stakeholders receive timely and reliable information to support efficient transportation management.

By providing features such as real-time GPS tracking, dynamic ETA calculation, intelligent route monitoring, automated passenger notifications, and operator alerts, RTBTS enhances the overall efficiency of the transport network. The system improves passenger convenience through accurate arrival predictions while supporting operators in maintaining schedule discipline and responding rapidly to delays, deviations, or emergencies. The architecture is supported by secure communication channels, scalable system components, and optimized data processing workflows, ensuring robustness and high performance even as user demand grows.

The development of RTBTS is guided by strong functional and non-functional requirements, emphasizing reliability, security, usability, and system responsiveness. This project contributes significantly to the modernization of public transportation by offering a dependable and intuitive solution that strengthens operational efficiency, reduces commuter uncertainty, and promotes trust between the transport authority and passengers. Ultimately, RTBTS is poised to elevate the quality of urban mobility by enabling smarter, safer, and more transparent public transport services.